

Клиентский аналитический сервис

Команда:

Аналитик Loginot · Пушкарева Арина Руслановна

Бэкенд-разработчик · Серебряков Всеволод Максимович

Фронтенд-разработчик · Фролов Фёдор Сергеевич

Задача и технологический стек

Задача

Собрать MVP личного кабинета клиента продуктового ретейлера. Пользователь вводит `user_id`, получает историю покупок, аналитические разрезы и текстовые рекомендации от языковой модели.

Данные

Открытый набор Instacart Market Basket Analysis (Kaggle): три таблицы - `orders`, `orders_prior`, `products`. База данных находится в сервисе Loginom



Технологический стек

Аналитика	Loginom (low-code)
Бэкенд	Python 3, Flask 3.x
LLM	Kimi K2.6/Ollama через NVIDIA NIM
Фронтенд	Jinja2, CSS, Chart.js
Деплой	Git, gunicorn, nginx

Loginom: шесть REST-сервисов в одном пакете

▼ ● instacart_ws_serebryakov_i_arina1	/ICT01/instacart_ws_serebryakov_i_arina1.lgp
 GetUserHistory	GetUserHistory
 CheckUserID	CheckUserID
 GetAnchorProducts	GetAnchorProducts
 GetPurchaseRhythm	GetPurchaseRhythm
 GetOrderTiming	GetOrderTiming
 GetCardOrder	GetCardOrder

```
21 def user_exists(user_id: int) -> bool:
22     rows = call_service("CheckUserID", user_id)
23     if not rows:
24         return False
25     return rows[0].get("user_exists", 0) == 1
26
27
28 def get_user_history(user_id: int) -> list[dict]:
29     return call_service("GetUserHistory", user_id)
30
31
32 def get_anchor_products(user_id: int) -> list[dict]:
33     return call_service("GetAnchorProducts", user_id)
34
35
36 def get_purchase_rhythm(user_id: int) -> dict:
37     rows = call_service("GetPurchaseRhythm", user_id)
38     return rows[0] if rows else {}
39
40
41 def get_order_timing(user_id: int) -> list[dict]:
42     return call_service("GetOrderTiming", user_id)
43
44
45 def get_cart_order(user_id: int) -> list[dict]:
46     return call_service("GetCardOrder", user_id)
```

1

CheckUserID

проверка существования user_id

2

GetUserHistory

топ-N товаров клиента с категорией

3

GetAnchorProducts

«якорные» товары (высокий reordered)

4

GetPurchaseRhythm

распределение интервалов между заказами

5

GetOrderTiming

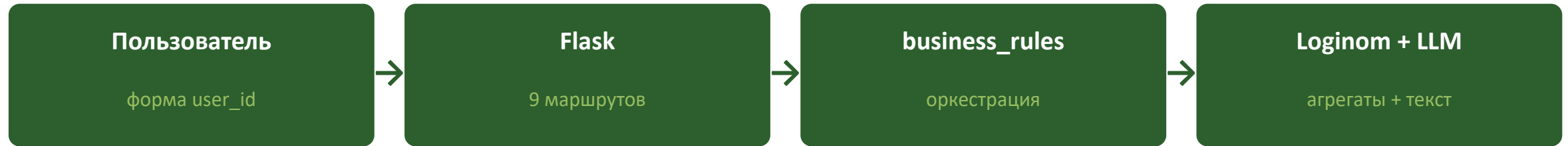
матрица «день недели × час»

6

GetCardOrder

приоритет vs импульс по add_to_cart_order

Flask-приложение и языковая модель



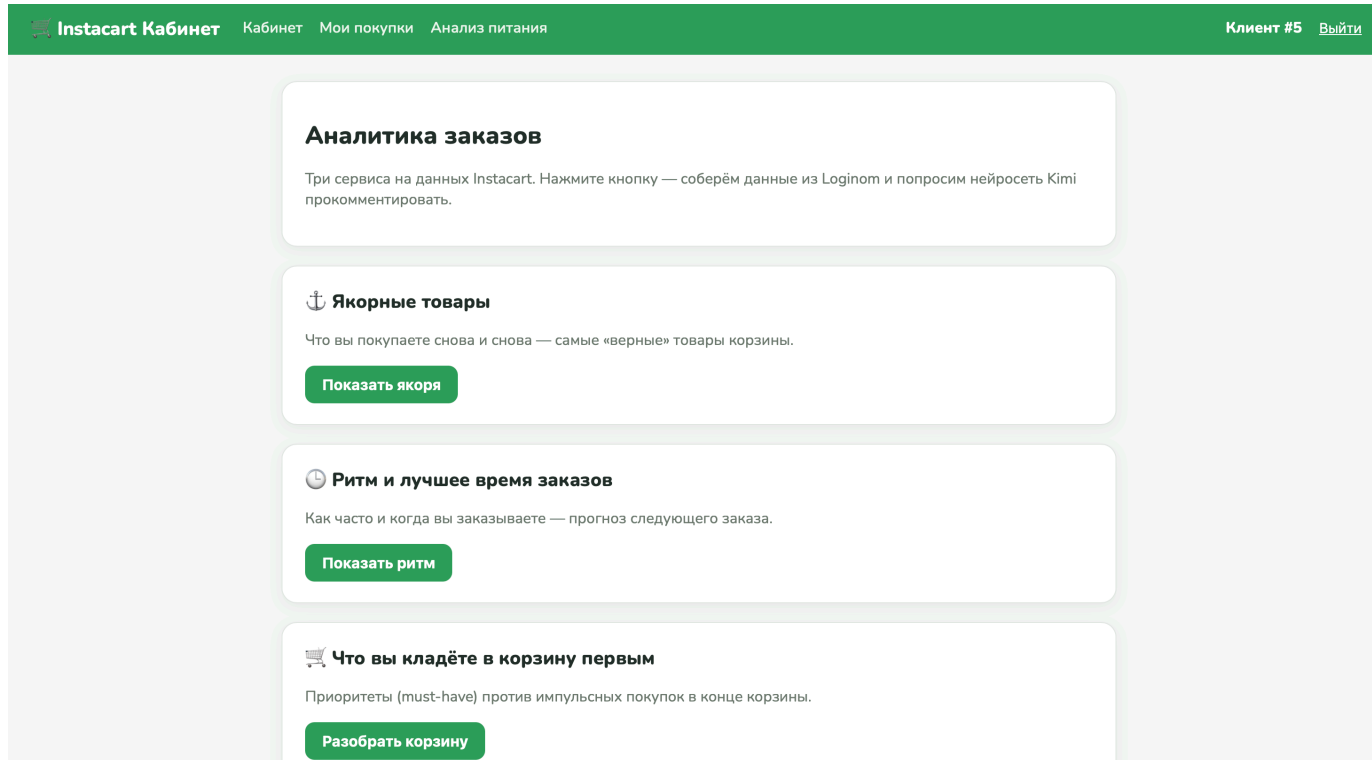
Архитектурные решения

- «Тонкие» маршруты: только сессия, вызов оркестратора и рендер
- Клиенты Loginom и LLM изолированы в отдельных модулях

Языковая модель

Модель	moonshotai/kimi-k2.6
Провайдер	NVIDIA NIM
Протокол	OpenAI-совместимый REST
temperature	0.3 (стабильные ответы)
max_tokens	800

Личный кабинет: витрина аналитики



Что реализовано

Карточная вёрстка

спокойная зелёно-белая палитра, единый стиль

Тепловая карта

CSS-сетка 7×24, без внешних библиотек

Chart.js по CDN

гистограммы под остальные сервисы

Индикатор кэша

серый значок «из кэша» / зелёный «свежий»

Кэширование ответов языковой модели

Ключ кэша

```
key = SHA-256(service | user_id | prompt)
```

Схема работы

- Клиент нажимает кнопку → бэкенд собирает промпт
- SHA-256 от ключа → поиск в таблице llm_cache
- Попадание: from_cache=True, ответ мгновенно из SQLite
- Проммах: запрос в Kimi K2.6, запись в кэш, from_cache=False

Индикатор в UI показывает пользователю, свежий ответ или из кэша

Зачем

×0

повторных обращений к модели при одинаковых входах

SHA-256

любое изменение промпта промахивается — свежий ответ

SQLite

переживает рестарт приложения

Тестирование и результаты

Автотесты

7 / 8

сценариев проходят зелёными

- Прогон ~10 секунд
- Работают с внешними запросами к Loginom
- Кэш проверяется счётчиком вызовов

План vs факт

2–3 доп. сервиса Loginom

4 сервиса

Витрина аналитики + LLM

выполнено

Кэш ответов модели

SQLite + индикатор

Автотесты

7/8 зелёных

Публикация в облаке

конфиги готовы

Выводы

1

Связка работает

Low-code аналитика + Flask + LLM даёт рабочий MVP силами команды из трёх человек за короткий срок.

2

Слой взаимозаменяемы

Модель меняется правкой двух строк конфига, аналитика — без остановки бэкенда, интерфейс — без переработки логики.

3

Роли естественны

Распределение «аналитик — бэкенд — фронтенд» точно ложится на цепочку «данные — оркестрация — интерфейс».

Спасибо за внимание